

SBI 811: Computer Programming in Bioinformatics

Lecture 5 — Functions, Debugging and Simple Testing

Duration: 3 hours

Level: Complete beginners

Prerequisite: variables, strings, numbers, conditions, lists and loops

Biological applications: reusable DNA cleaning, validation, GC content, reverse complement and small sequence-analysis pipelines

Work slowly. Read each explanation, predict the result, run the cell, and compare the result with your prediction.

Learning outcomes

By the end of this lecture, you should be able to:

1. explain why programmers divide a program into functions;
2. define and call a function;
3. distinguish a **parameter** from an **argument**;
4. distinguish `print()` from `return`;
5. write functions with one or more parameters;
6. use a default parameter appropriately;
7. understand local variables at an introductory level;
8. document a function using a docstring;
9. read the final line of a traceback to identify an error;
10. use small checks and `assert` statements to test a function;
11. consider normal, boundary and unusual inputs; and
12. combine small functions into a simple DNA-analysis workflow.

File reading, dictionaries, pandas, classes and testing frameworks are intentionally postponed.

Part 1 — Why functions?

A **function** is a named block of code designed to perform one task. You have already used functions written by other programmers:

- `print()` displays information;
- `len()` counts items;
- `type()` reports a value's data type;

- `.upper()` creates an uppercase string; and
- `.count()` counts occurrences.

Today you will write your own functions.

```
In [ ]: sequence = "ATGCGTAA"
print(sequence)
print(len(sequence))
print(type(sequence))
```

1.1 The problem with repeated code

Suppose we must calculate GC percentage for three sequences. We could copy the calculation three times, but copied code is longer and easier to make inconsistent.

```
In [ ]: sequence_1 = "ATGCGC"
gc_1 = sequence_1.count("G") + sequence_1.count("C")
percentage_1 = (gc_1 / len(sequence_1)) * 100

sequence_2 = "AAAAAT"
gc_2 = sequence_2.count("G") + sequence_2.count("C")
percentage_2 = (gc_2 / len(sequence_2)) * 100

sequence_3 = "GGCCGG"
gc_3 = sequence_3.count("G") + sequence_3.count("C")
percentage_3 = (gc_3 / len(sequence_3)) * 100

print(percentage_1, percentage_2, percentage_3)
```

A function lets us write the calculation once and reuse it. Benefits include:

- **reuse** — call the same code for many sequences;
- **clarity** — a good function name explains the task;
- **maintenance** — correct the logic in one place; and
- **testing** — check one small task independently.

Core idea: one small function should usually do one clear job.

Part 2 — Defining and calling a function

The smallest useful pattern is:

```
def function_name():
    statement
```

- `def` means “define a function”.
- The name follows the same rules as a variable name.
- Parentheses follow the name.
- The header ends with a colon.
- The body is indented, normally by four spaces.

Running a definition stores the function; it does **not** run the body.

```
In [ ]: def show_welcome():  
        print("Welcome to SBI 811")  
        print("Today we will build reusable sequence tools.")
```

Nothing was displayed because the function has only been defined. To run it, **call** it using its name followed by parentheses.

```
In [ ]: show_welcome()
```

A function can be called more than once.

```
In [ ]: show_welcome()  
show_welcome()
```

Checkpoint 1

Predict what happens in each case:

1. Run only a function definition.
2. Write the function name without parentheses.
3. Call the function with parentheses.

```
In [ ]: # Try the function name without parentheses.  
show_welcome
```

Jupyter displays a description such as `<function ...>`. Parentheses are what request execution.

Guided practice 1 — A sequence heading

Complete a function named `show_sequence_heading()` that prints `DNA sequence report`.

```
In [ ]: def show_sequence_heading():  
        # Replace the next line with a print statement.  
        pass  
  
        # Remove the # after completing the function.  
        # show_sequence_heading()
```

Guided practice 1 — solution

```
In [ ]: def show_sequence_heading():  
        print("DNA sequence report")  
  
show_sequence_heading()
```

Part 3 — Parameters and arguments

A function becomes more useful when it receives data.

```
def show_length(sequence):  
    print(len(sequence))
```

- `sequence` in the definition is a **parameter**: a placeholder name.
- `"ATGC"` in `show_length("ATGC")` is an **argument**: the actual value supplied.

Think of a parameter as a labelled empty box and an argument as the item placed in it.

```
In [ ]: def show_length(sequence):  
        print("Sequence:", sequence)  
        print("Length:", len(sequence))  
  
        show_length("ATGC")  
        show_length("GGCCAATT")
```

The parameter exists so the same instructions can work with different arguments.

```
In [ ]: dna = "ATGCGTAA"  
        show_length(dna)
```

3.1 Meaningful names

Prefer a name that states the task:

```
def show_length(sequence):
```

Avoid vague names such as:

```
def do_it(x):
```

Common Python style uses lowercase words separated by underscores:

```
calculate_gc_percentage.
```

Guided practice 2 — Count one base

Write `show_base_count(sequence, base)` using two parameters. It should print the base and its count.

```
In [ ]: def show_base_count(sequence, base):  
        # Write one print statement here.  
        pass  
  
        # show_base_count("ATGATA", "A")
```

Guided practice 2 — solution

```
In [ ]: def show_base_count(sequence, base):  
        print(base, "occurs", sequence.count(base), "times")  
  
        show_base_count("ATGATA", "A")  
        show_base_count("ATGATA", "G")
```

Part 4 — `print()` and `return` are different

This distinction is essential.

- `print()` displays a value for a person to see.
- `return` sends a value back to the part of the program that called the function.

A printed value is not automatically available for a later calculation.

```
In [ ]: def print_length(sequence):  
        print(len(sequence))  
  
        printed_result = print_length("ATGCGT")  
        print("Stored value:", printed_result)
```

The function displayed `6`, but `printed_result` contains `None`. `None` means no useful value was returned.

```
In [ ]: def get_length(sequence):  
        return len(sequence)  
  
        returned_result = get_length("ATGCGT")  
        print("Stored value:", returned_result)  
        print("Twice the length:", returned_result * 2)
```

`return` makes the result reusable. We can store it, compare it, print it or pass it to another function.

```
In [ ]: length = get_length("ATGCGT")  
  
        if length >= 6:  
            print("The sequence meets the minimum length.")
```

4.1 Code after `return`

When Python reaches `return`, the function ends immediately. Statements after that point in the same path are not run.

```
In [ ]: def demonstrate_return():  
        print("Before return")  
        return "finished"  
        print("After return")  
  
        message = demonstrate_return()  
        print(message)
```

Checkpoint 2 — Predict first

```
In [ ]: def double(number):  
        return number * 2
```

```
answer = double(7)
print(answer)
print(double(3) + 1)
```

Expected output:

```
14
7
```

Part 5 — Building biological functions

We will build several small tools. Each solves one task, making mistakes easier to locate.

5.1 Clean a sequence

Sequence text may contain surrounding spaces, a newline from copied text, or lowercase letters. `strip()` removes whitespace from both ends; `upper()` standardizes letter case.

The newline below imitates the line ending commonly obtained when text is read from a file.

```
In [ ]: raw_dna = " atGCGtaa\n"
        print(repr(raw_dna))

        def clean_sequence(sequence):
            cleaned = sequence.strip().upper()
            return cleaned

        cleaned_dna = clean_sequence(raw_dna)
        print(repr(cleaned_dna))
```

`repr()` is used here only to make hidden characters such as ``` visible. The cleaning function itself works without `repr()`.

`strip()` removes whitespace from both ends. `rstrip()` removes it only from the right end. For cleaning an entire sequence line, `strip()` is usually appropriate.

5.2 Count a requested nucleotide

This function returns a count instead of printing it.

```
In [ ]: def count_nucleotide(sequence, nucleotide):
        cleaned_sequence = sequence.strip().upper()
        cleaned_nucleotide = nucleotide.upper()
        return cleaned_sequence.count(cleaned_nucleotide)

        adenine_count = count_nucleotide("atgaaa", "a")
        print("A count:", adenine_count)
```

5.3 Calculate GC percentage

The formula is:

$$GC\% = \frac{G + C}{\text{sequence length}} \times 100$$

We must decide what should happen for an empty sequence because division by zero is impossible.

```
In [ ]: def calculate_gc_percentage(sequence):
        cleaned_sequence = sequence.strip().upper()

        if len(cleaned_sequence) == 0:
            return 0.0

        gc_count = cleaned_sequence.count("G") + cleaned_sequence.count("C")
        gc_percentage = (gc_count / len(cleaned_sequence)) * 100
        return gc_percentage

        print(calculate_gc_percentage("ATGCGC"))
        print(calculate_gc_percentage("AAAAAT"))
        print(calculate_gc_percentage(""))
```

Returning `0.0` for an empty sequence is a design decision suitable for this beginner exercise. In scientific software, the team should document such decisions clearly.

5.4 Round only for presentation

Keep the full numeric result for calculations. Round when displaying it.

```
In [ ]: gc_result = calculate_gc_percentage("ATGCGC")
        print("Full result:", gc_result)
        print("Displayed result:", round(gc_result, 2), "%")
```

Guided practice 3 — AT percentage

Write a function named `calculate_at_percentage(sequence)`. It should:

1. clean the sequence;
2. return `0.0` if it is empty;
3. count `A` and `T`;
4. calculate the percentage; and
5. return the result.

```
In [ ]: def calculate_at_percentage(sequence):
        # Write your solution here.
        pass

        # print(calculate_at_percentage("AAATGC"))
```

Guided practice 3 — solution

```
In [ ]: def calculate_at_percentage(sequence):
        cleaned_sequence = sequence.strip().upper()
        if len(cleaned_sequence) == 0:
            return 0.0
        at_count = cleaned_sequence.count("A") + cleaned_sequence.count("T")
        return (at_count / len(cleaned_sequence)) * 100

print(calculate_at_percentage("AAATGC"))
```

Part 6 — Functions can call other functions

We do not need to repeat cleaning logic in every function. One function may call another.

```
In [ ]: def calculate_gc_percentage_v2(sequence):
        cleaned_sequence = clean_sequence(sequence)
        if len(cleaned_sequence) == 0:
            return 0.0
        gc_count = cleaned_sequence.count("G") + cleaned_sequence.count("C")
        return (gc_count / len(cleaned_sequence)) * 100

print(calculate_gc_percentage_v2(" atgcgc\n"))
```

This creates a simple flow:

raw sequence → clean_sequence() → calculate_gc_percentage_v2() → numeric result

Small functions can become building blocks for a larger analysis.

6.1 Validate DNA

For this lecture, a valid DNA sequence contains only `A`, `T`, `G` and `C`. We will return `True` or `False`.

```
In [ ]: def is_valid_dna(sequence):
        cleaned_sequence = clean_sequence(sequence)

        if len(cleaned_sequence) == 0:
            return False

        for base in cleaned_sequence:
            if base not in "ATGC":
                return False

        return True

print(is_valid_dna("ATGCGT"))
print(is_valid_dna("ATGXGT"))
print(is_valid_dna(""))
```

Notice the placement of the final `return True`. Python can report success only after the loop has checked **every** base.

Common logic error — returning too early

This incorrect version checks only the first base:

```
def is_valid_dna_wrong(sequence):
    for base in sequence:
        if base not in "ATGC":
            return False
    return True    # wrongly inside the loop
```

For "AX", it sees A, immediately returns True, and never examines X. Indentation changes meaning.

6.2 Classify GC percentage

```
In [ ]: def classify_gc(sequence):
        gc_percentage = calculate_gc_percentage_v2(sequence)

        if gc_percentage >= 60:
            return "high GC"
        elif gc_percentage >= 40:
            return "moderate GC"
        else:
            return "low GC"

print(classify_gc("GGCCGG"))
print(classify_gc("ATGCAT"))
print(classify_gc("AAAAAT"))
```

Part 7 — Multiple and default parameters

A function may receive more than one argument.

```
In [ ]: def sequence_meets_length(sequence, minimum_length):
        cleaned_sequence = clean_sequence(sequence)
        return len(cleaned_sequence) >= minimum_length

print(sequence_meets_length("ATGCGT", 5))
print(sequence_meets_length("ATG", 5))
```

Arguments are normally matched to parameters by position. In

sequence_meets_length("ATG", 5), "ATG" becomes sequence, and 5 becomes minimum_length.

7.1 Default parameter values

A default supplies a value when the caller does not provide one.

```
In [ ]: def sequence_meets_length_v2(sequence, minimum_length=5):
        cleaned_sequence = clean_sequence(sequence)
        return len(cleaned_sequence) >= minimum_length

print(sequence_meets_length_v2("ATGCGT"))    # uses 5
print(sequence_meets_length_v2("ATGCGT", 10)) # uses 10
```

Use defaults only when there is a sensible, clearly documented usual value. Required parameters come before default parameters.

7.2 Keyword arguments

Naming an argument can improve readability.

```
In [ ]: print(sequence_meets_length_v2(sequence="ATGCGT", minimum_length=4))
```

Part 8 — Local variables and scope

A variable created inside a function is normally **local** to that function. It exists while the function runs and helps prevent unrelated parts of a program from interfering with one another.

```
In [ ]: def calculate_length(sequence):
        sequence_length = len(sequence)
        return sequence_length

result = calculate_length("ATGC")
print(result)
```

`sequence_length` is local. Trying to use it outside the function would cause `NameError`. The returned value is stored outside as `result`.

Avoid relying on global variables when a function can receive an argument instead.

```
In [ ]: dna_sequence = "ATGC"

def unclear_length():
    return len(dna_sequence) # depends on an outside variable

def clear_length(sequence):
    return len(sequence)    # receives its dependency explicitly

print(unclear_length())
print(clear_length("ATGC"))
```

Part 9 — Docstrings: explaining a function

A **docstring** is a short description placed immediately inside a function using triple quotes. It should state what the function does, not narrate every line.

```
In [ ]: def calculate_gc_with_docstring(sequence):
        """Return the GC percentage of a DNA sequence."""
        cleaned_sequence = clean_sequence(sequence)
        if len(cleaned_sequence) == 0:
            return 0.0
        gc_count = cleaned_sequence.count("G") + cleaned_sequence.count("C")
        return (gc_count / len(cleaned_sequence)) * 100

help(calculate_gc_with_docstring)
```

For beginner functions, one clear sentence is often enough. A longer docstring may describe parameters, the returned value and special cases.

```
In [ ]: def count_nucleotide_documented(sequence, nucleotide):
        """Count one nucleotide in a DNA sequence.

        sequence: DNA text to examine.
        nucleotide: one base such as A, T, G or C.
        Returns: an integer count.
        """
        cleaned_sequence = clean_sequence(sequence)
        return cleaned_sequence.count(nucleotide.upper())

print(count_nucleotide_documented("ATGATA", "a"))
```

Part 10 — Reverse complement as a function

DNA strands are antiparallel. A useful operation is to:

1. replace each base with its complement (A↔T , G↔C); then
2. reverse the complemented sequence.

We will use conditions and a loop, avoiding dictionaries until a later lesson.

```
In [ ]: def reverse_complement(sequence):
        """Return the reverse complement of a valid DNA sequence."""
        cleaned_sequence = clean_sequence(sequence)
        complement = ""

        for base in cleaned_sequence:
            if base == "A":
                complement = complement + "T"
            elif base == "T":
                complement = complement + "A"
            elif base == "G":
                complement = complement + "C"
            elif base == "C":
                complement = complement + "G"
            else:
                return "Invalid DNA sequence"

        return complement[::-1]

print(reverse_complement("ATGC"))
print(reverse_complement("atgc"))
print(reverse_complement("ATGX"))
```

For `ATGC` :

- complement: `TACG`
- reverse complement: `GCAT`

The slice `[::-1]` makes a reversed copy of the string.

Guided practice 4 — Explain the function

In your own words, answer:

1. Why is `clean_sequence()` called first?
2. Why does the loop use `if / elif`?
3. Why is the string reversed at the end?
4. What is returned for an invalid character?

Part 11 — Debugging: finding and correcting errors

Debugging is the systematic process of finding and correcting a problem. An error is evidence about what Python could not do; it is not evidence that you cannot program.

Use this routine:

1. read the final line of the traceback;
2. identify the error type and message;
3. find the referenced line;
4. inspect names, types, punctuation and indentation;
5. make one small correction; and
6. run the smallest relevant test again.

11.1 Reading a traceback

A traceback is read mostly from the bottom. For example:

```
NameError: name 'sequence' is not defined
```

The type is `NameError`; the message points to a misspelled name. Error demonstrations below are caught so this teaching notebook can still run from top to bottom.

```
In [ ]: try:
        print(sequence)
except NameError as error:
    print("Caught error:", type(error).__name__)
    print("Message:", error)
```

11.2 Common error types

Error	Usual meaning	First thing to inspect
<code>NameError</code>	A name does not exist	spelling; whether a defining cell ran
<code>TypeError</code>	Operation used incompatible types	value types and function arguments
<code>ZeroDivisionError</code>	Division by zero	empty data or zero denominator
<code>SyntaxError</code>	Python cannot parse the code	colon, brackets, quotes, spelling

Error	Usual meaning	First thing to inspect
<code>IndentationError</code>	Indentation is inconsistent or missing	spaces below <code>def</code> , <code>if</code> , or loop
<code>AssertionError</code>	A test expectation was not met	function result or expected value

TypeError example

```
In [ ]: try:
        print("Length: " + 6)
    except TypeError as error:
        print("Caught error:", type(error).__name__)
        print("Message:", error)

    # Two correct alternatives:
    print("Length:", 6)
    print("Length: " + str(6))
```

ZeroDivisionError example

```
In [ ]: def unsafe_gc(sequence):
        gc_count = sequence.count("G") + sequence.count("C")
        return (gc_count / len(sequence)) * 100

    try:
        unsafe_gc("")
    except ZeroDivisionError as error:
        print("Caught error:", type(error).__name__)
        print("Message:", error)
```

The corrected function checks for an empty sequence before division, as `calculate_gc_percentage()` already does.

11.3 Syntax and indentation errors

These examples are shown as text because a real syntax error would prevent the cell from running.

Missing colon:

```
def show_length(sequence)
    print(len(sequence))
```

Missing indentation:

```
def show_length(sequence):
print(len(sequence))
```

Correct:

```
def show_length(sequence):
    print(len(sequence))
```

11.4 Logic errors

A logic error runs without crashing but gives a wrong result. This makes it especially important to know the expected answer.

```
In [ ]: def gc_percentage_wrong(sequence):
        gc_count = sequence.count("G") # C was accidentally omitted
        return (gc_count / len(sequence)) * 100

print("Wrong result:", gc_percentage_wrong("GGCC"))
print("Expected result: 100.0")
```

11.5 Debugging with temporary `print()` statements

Temporary prints reveal intermediate values.

```
In [ ]: def gc_percentage_debug(sequence):
        cleaned_sequence = clean_sequence(sequence)
        print("DEBUG cleaned:", repr(cleaned_sequence))
        gc_count = cleaned_sequence.count("G") + cleaned_sequence.count("C")
        print("DEBUG GC count:", gc_count)
        print("DEBUG length:", len(cleaned_sequence))
        return (gc_count / len(cleaned_sequence)) * 100

print("Result:", gc_percentage_debug(" atgcgcn"))
```

Remove or disable debugging prints after the problem is solved so scientific output remains clear.

Debugging practice — Find three problems

Do not run this broken version. Identify the mistakes first.

```
def calculate_gc(sequence)
cleaned = sequence.strip.upper()
    gc = cleaned.count("G") + cleaned.count("C")
    return gc / len(cleaned) * 100
```

Hints: inspect the function header, indentation and method calls.

Debugging practice — solution

1. Add `:` after `calculate_gc(sequence)`.
2. Indent the function body.
3. Call `strip` using parentheses: `strip()`.

An empty-sequence check is also needed for robust behaviour.

```
In [ ]: def calculate_gc_corrected(sequence):
        cleaned = sequence.strip().upper()
        if len(cleaned) == 0:
            return 0.0
        gc = cleaned.count("G") + cleaned.count("C")
        return (gc / len(cleaned)) * 100
```

```
print(calculate_gc_corrected("ATGCGC"))
```

Part 12 — Simple testing

Testing asks: **Does the function produce the expected result for known inputs?**

Start with examples whose correct answers you can calculate manually.

```
In [ ]: actual = calculate_gc_percentage_v2("GGCC")
        expected = 100.0

        print("Actual:", actual)
        print("Expected:", expected)
        print("Test passed:", actual == expected)
```

12.1 assert

`assert` states that a condition must be true. If it is true, nothing happens. If false, Python raises `AssertionError`.

```
In [ ]: assert calculate_gc_percentage_v2("GGCC") == 100.0
        assert calculate_gc_percentage_v2("AAAA") == 0.0
        assert calculate_gc_percentage_v2("ATGC") == 50.0

        print("All GC tests passed.")
```

A useful message can explain the expectation.

```
In [ ]: assert clean_sequence(" atgc\n") == "ATGC", "Cleaning should strip whitespace"
        print("Cleaning test passed.")
```

12.2 Test categories

Do not test only one comfortable input.

Category	Example	Why test it?
Typical	ATGC	ordinary use
All GC	GGCC	expected 100%
No GC	AAAA	expected 0%
Lowercase	atgc	cleaning behaviour
Whitespace	`" ATGC`	

```
" | copied/file-like text |
| Empty | "" | boundary case and division by zero |
| Invalid | ATGX` | validation behaviour |
```

```
In [ ]: assert clean_sequence(" atgc\n") == "ATGC"
        assert is_valid_dna("ATGC") is True
```

```

assert is_valid_dna("ATGX") is False
assert is_valid_dna("") is False
assert reverse_complement("ATGC") == "GCAT"
assert reverse_complement("atgc") == "GCAT"

print("Sequence-tool tests passed.")

```

Guided practice 5 — Design tests

Before writing code, state the expected results for `count_nucleotide()`:

1. `count_nucleotide("AAAA", "A")`
2. `count_nucleotide("ATGC", "G")`
3. `count_nucleotide("atga", "A")`
4. `count_nucleotide("", "C")`

Then write four assertions.

```
In [ ]: # Write your four assertions here.
```

Guided practice 5 — solution

```
In [ ]: assert count_nucleotide("AAAA", "A") == 4
assert count_nucleotide("ATGC", "G") == 1
assert count_nucleotide("atga", "A") == 2
assert count_nucleotide("", "C") == 0
print("Nucleotide-count tests passed.")
```

Part 13 — Integrated DNA-analysis workflow

We now combine small, tested functions to analyse several short sequences. The workflow will:

1. clean each sequence;
2. validate it;
3. calculate length and GC percentage;
4. classify its GC content; and
5. calculate its reverse complement.

```
In [ ]: sample_sequences = [
    " atgcgc\n",
    "AAAAAT",
    "GGCCGG",
    "ATGXTA"
]

for raw_sequence in sample_sequences:
    sequence = clean_sequence(raw_sequence)
    print("Sequence:", sequence)

    if is_valid_dna(sequence):
        print("Valid DNA: Yes")
```

```

print("Length:", get_length(sequence))
print("GC percentage:", round(calculate_gc_percentage_v2(sequence),
print("GC class:", classify_gc(sequence))
print("Reverse complement:", reverse_complement(sequence))
else:
    print("Valid DNA: No – analysis skipped")

print("-" * 35)

```

Why this design is easier to trust

The loop coordinates the workflow, while each function has one job:

Function	Responsibility
<code>clean_sequence()</code>	standardize sequence text
<code>is_valid_dna()</code>	accept or reject DNA symbols
<code>get_length()</code>	return sequence length
<code>calculate_gc_percentage_v2()</code>	calculate GC percentage
<code>classify_gc()</code>	interpret GC percentage
<code>reverse_complement()</code>	construct reverse complement

If a reverse complement is wrong, we can examine that function without searching through one very large block.

Part 14 — AI-assisted challenge

Use an AI assistant as a **reviewer**, not as a replacement for your reasoning.

Task

Create a function named `sequence_report(sequence, minimum_length=5)` that returns one sentence. It should:

1. clean the sequence;
2. reject empty or invalid DNA;
3. calculate length and GC percentage;
4. state whether the minimum length is met; and
5. include the reverse complement.

Example style:

```

Length=6; GC=50.0%; minimum length met=True; reverse
complement=GCATGC

```

Responsible AI workflow

1. Write pseudocode yourself.
2. Ask the AI to critique the pseudocode, not immediately replace it.

3. Implement a first version.
4. Ask the AI to suggest edge cases.
5. Test every suggestion with inputs whose expected result you understand.
6. Record which suggestion you accepted and why.

Never trust code only because it runs. A program can run and still be biologically or logically wrong.

Challenge starter

```
In [ ]: def sequence_report(sequence, minimum_length=5):
        """Return a short report for a DNA sequence."""
        # 1. Clean the sequence.
        # 2. Validate it.
        # 3. Calculate the requested values.
        # 4. Return one report string.
        pass

        # Add at least five tests after completing the function.
```

Challenge solution — inspect only after attempting

```
In [ ]: def sequence_report(sequence, minimum_length=5):
        """Return a short report for a DNA sequence."""
        cleaned_sequence = clean_sequence(sequence)

        if not is_valid_dna(cleaned_sequence):
            return "Invalid DNA sequence"

        length = len(cleaned_sequence)
        gc_percentage = calculate_gc_percentage_v2(cleaned_sequence)
        meets_minimum = length >= minimum_length
        reverse_comp = reverse_complement(cleaned_sequence)

        return (
            f"Length={length}; GC={gc_percentage:.1f}%; "
            f"minimum length met={meets_minimum}; "
            f"reverse complement={reverse_comp}"
        )

        print(sequence_report("ATGCAT"))
        print(sequence_report("ATGXAT"))
```

```
In [ ]: assert sequence_report("ATGX") == "Invalid DNA sequence"
        assert sequence_report("") == "Invalid DNA sequence"
        assert "Length=4" in sequence_report("ATGC")
        assert "GC=50.0%" in sequence_report("ATGC")
        assert "minimum length met=False" in sequence_report("ATGC", 5)
        assert "reverse complement=GCAT" in sequence_report("ATGC")
        print("Challenge tests passed.")
```

Part 15 — Independent practice

Exercise A — contains_motif

Write `contains_motif(sequence, motif)` that cleans both inputs and returns `True` when the motif occurs in the sequence. Test:

- motif present;
- motif absent;
- lowercase input; and
- empty motif. Decide and document what an empty motif should mean.

```
In [ ]: # Your contains_motif() function and tests go here.
```

Exercise B — `count_unknown_bases`

Write a function that counts `N` symbols in a sequence. It should handle lowercase input. Test at least three cases.

```
In [ ]: # Your count_unknown_bases() function and tests go here.
```

Exercise C — Debug a student's function

The intention is to return the percentage of `N` bases:

```
def n_percentage(sequence):  
    cleaned = sequence.strip().upper()  
    n_count = cleaned.count("N")  
    percentage = n_count / len(sequence) * 100  
    print(percentage)
```

Questions:

1. What happens for an empty sequence?
2. Why should the denominator use `len(cleaned)` ?
3. Why might `return` be more useful than `print()` ?
4. Write a corrected function and four tests.

```
In [ ]: # Your corrected n_percentage() function and tests go here.
```

Part 16 — Formative review

Answer without running code first.

1. What is the purpose of a function?
2. What is the difference between a parameter and an argument?
3. What is the difference between `print()` and `return` ?
4. Why is indentation important inside a function?
5. What does `None` often indicate after calling a function?
6. Why should a function usually perform one clear task?
7. Which part of a traceback should you read first?
8. What is a logic error?

9. What does `assert` do?
10. Name three types of input that should be tested.
11. Why should invalid DNA be detected before further analysis?
12. Why should expected results be calculated independently of the code being tested?

Review answers

1. A function names and packages reusable instructions for a task.
2. A parameter is the placeholder in the definition; an argument is an actual supplied value.
3. `print()` displays; `return` sends a reusable value back.
4. Indentation tells Python which statements belong to the function.
5. The function probably did not return a value explicitly.
6. Small focused functions are easier to understand, reuse, debug and test.
7. Begin with the final line: error type and message.
8. Code runs but produces an incorrect result.
9. It checks that a condition is true and raises `AssertionError` if false.
10. Typical, boundary/empty, lowercase/whitespace, and invalid inputs are examples.
11. Invalid symbols can make later biological results misleading.
12. Otherwise the same mistaken reasoning may be used in both the code and the expected answer.

Part 17 — Take-home tracked exercise

Create a new notebook named:

`SBI811_Lecture5_Exercise_YourName.ipynb`

It must contain:

1. `clean_sequence(sequence)` ;
2. `is_valid_dna(sequence)` ;
3. `calculate_gc_percentage(sequence)` ;
4. one additional function of your choice;
5. a docstring for every function;
6. at least three tests per function;
7. one short Markdown paragraph explaining a bug you encountered and how you corrected it; and
8. a final code cell that analyses at least four sequences, including one invalid sequence.

Track the notebook in your own GitHub repository:

```
git status
git add SBI811_Lecture5_Exercise_YourName.ipynb
git commit -m "Complete SBI 811 Lecture 5 exercise"
git push
```

Do not push exercise work directly to the shared class repository unless the instructor explicitly asks you to do so.

Lecture summary

You can now:

- define and call functions;
- pass arguments through parameters;
- return reusable results;
- combine small biological functions;
- document code with docstrings;
- read common error messages;
- debug using small, controlled checks;
- test expected behaviour using `assert` ; and
- build a small, reusable DNA-analysis workflow.

The next lecture can use these functions while introducing biological text files such as FASTA.